BudgetNAS: Continual Online Neural Architecture Search with Budgeted Architecture Mutation

Anonymous CVPR-NAS26 Submission

Abstract

Deploying deep learning models in non-stationary environments requires architectures that can adapt to distribution shifts. Existing NAS methods assume fixed data distributions, while continual learning methods operate on fixed architectures. We propose **BudgetNAS**, a framework for *continual online NAS* that mutates a deployed model's architecture under a strict per-domain mutation budget B. We evaluate ten methods—spanning fixed-architecture baselines, naive growth strategies, continual learning methods, and three BudgetNAS variants—on a three-domain streaming benchmark (CIFAR-10 $\rightarrow$ CIFAR-100 $\rightarrow$ SVHN) across three random seeds. Our recommended configuration, BudgetNAS-Heuristic with B=1, achieves the best CIFAR-10 accuracy (42.7$\pm$0.7%), competitive CIFAR-100 (11.6$\pm$0.5%), and strong SVHN (35.6$\pm$1.3%) with only 1.47M parameters—the highest accuracy-per-parameter efficiency among all architecture-changing methods. Comprehensive ablations reveal three unexpected findings: (1) stabilization techniques (warm-start, layer freezing) *hurt* rather than help; (2) drift detection is unnecessary—all useful mutations are triggered by low accuracy alone; and (3) a single well-placed mutation per domain (B=1) outperforms larger budgets. We additionally validate on gradual shifts and a four-domain long stream.

1 Introduction

Neural Architecture Search (NAS) [25, 7] has achieved remarkable success in automatically discovering high-performance architectures, but nearly all NAS methods assume a static data distribution. In practice, deployed models encounter *distribution shift*—data characteristics change over time due to evolving domains, user populations, or environmental conditions. When shift occurs, a fixed architecture may become suboptimal, yet re-running NAS from scratch is prohibitively expensive.

Continual learning [9, 4, 23] addresses non-stationarity by adapting model *parameters*, but typically keeps the architecture fixed. A few works have explored architectural adaptation [10, 18], but these assume unlimited growth or do not impose realistic constraints on the architectural search budget.

We argue that a practical online NAS system must satisfy **controlled growth** (mutations are expensive and must be limited), **selective mutation** (not all shifts require architectural changes), and **bidirectional adaptation** (both grow and prune).

We propose **BudgetNAS**, a framework combining a modular neural network with a *budgeted mutation controller*. Given a task-incremental stream with known domain boundaries, BudgetNAS selectively applies architecture mutations—adding blocks, inserting downsampling stages, or pruning layers—while respecting a per-domain budget of at most B mutations.

Our approach and key findings. We evaluate three BudgetNAS controller variants (heuristic, heuristic with stabilization, and Thompson sampling bandit) against seven baselines. The main result is that a simple heuristic controller with *performance-triggered* mutation and B=1 achieves the best tradeoff between accuracy and model size. Along the way, we discover that several intuitively appealing design choices—drift detection, warm-start initialization, layer freezing, learned bandit controllers—either provide no benefit or actively hurt performance at this scale. We report these negative results honestly, as they provide actionable guidance for practitioners.

Our contributions:

- A *continual online NAS* framework with explicit mutation budgets, bridging NAS and continual learning with deployment realism.
- Evaluation of ten methods across three seeds with five ablation studies (budget sweep, trigger ablation, stabilization ablation, gradual shift, long stream).
- Honest analysis of negative results: stabilization hurts, drift detection is unnecessary, and the bandit controller underperforms the heuristic at low budgets.
- A practical deployment recommendation: use performance-triggered heuristic mutation with B=1 for the best efficiency–accuracy tradeoff.

2 Related Work

Neural Architecture Search. Modern NAS ranges from reinforcement learning [25, 14] and evolutionary approaches [6] to differentiable methods [11, 2]. Elastic architectures [1, 24] train a single supernet from which sub-networks are extracted. Network morphism [19, 5] enables function-preserving transformations. Hardware-aware NAS [15, 21, 22, 16] optimizes under deployment constraints. None address *online* adaptation to non-stationary data streams.

Continual Learning. Regularization methods such as EWC [9] penalize changes to important parameters. Replay methods [4, 12] store exemplars. These operate on fixed architectures.

Continual Learning with Architectural Changes. DEN [23] expands capacity via selective retraining. PackNet [13] allocates sub-networks through pruning. Learn to Grow [10] frames continual learning as structure selection. Modular networks [18], Supermasks [20], and NISPA [8] explore task-specific capacity allocation. These methods generally assume unlimited growth or do not impose per-transition budgets.

Our contribution: BudgetNAS uniquely combines task-incremental streaming, explicit per-domain mutation budgets, bidirectional mutation operators, and principled controller comparison (heuristic vs. learned).

3 Method

3.1 Overview

Figure 1 illustrates the BudgetNAS framework. A modular neural network processes a stream of data chunks from different domains. Domain boundaries are known and each domain receives a fresh classifier head. Within each domain, a *BudgetNAS Controller* monitors performance and proposes architecture mutations subject to a per-domain budget B.

3.2 Modular Network Architecture

We define a modular CNN composed of a fixed stem layer followed by mutable ResBlocks $\{b_1, \ldots, b_K\}$ and a task-specific classifier head. Each ResBlock contains two 3×3 convolutions with batch normalization, ReLU activations, and a skip connection. Three mutation operators modify the block sequence:

- **AddBlock**: Appends a ResBlock with matching channel dimensions.
- **AddDownsample**: Appends a stride-2 ResBlock, doubling channels (up to 256), increasing the receptive field.

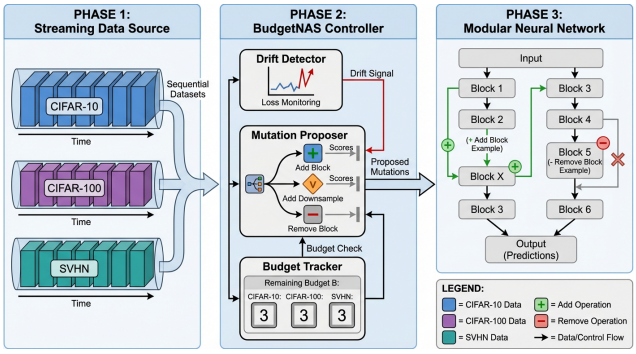

Figure 1: **BudgetNAS Framework.** Data arrives in a task-incremental stream (left). The BudgetNAS Controller (center) monitors accuracy and proposes mutations under budget B. The modular network (right) is mutated selectively.

- **RemoveBlock**: Removes the second-to-last block (preserving the classifier interface).

When a new domain begins, the classifier head is replaced for the target classes. Existing feature extraction layers are preserved for transfer.

3.3 BudgetNAS Controllers

We investigate two controller designs:

Heuristic Controller. The controller maintains a per-domain budget B that resets at each domain transition. At the first chunk of each new domain (and any chunk where accuracy falls below threshold $\gamma = 0.35$), the controller proposes the highest-scoring feasible mutation:

$$s(\text{mut}) = s_{\text{base}}(\text{mut}) + \beta \cdot \mathbb{1}[\text{acc} < \gamma] \tag{1}$$

where s_{base} assigns priors (AddBlock = 0.5, AddDownsample = 0.4, RemoveBlock = 0.2) and $\beta = 0.2$ is a low-accuracy bonus. The mutation is skipped if the budget is exhausted or accuracy exceeds γ.

Bandit Controller. We also test a Thompson sampling [17, 3] contextual bandit that maintains Beta(α_a, β_a) priors for each action $a \in \{\text{AddBlock, AddDownsample, RemoveBlock, NoOp}\}$. At each mutation opportunity, the controller samples from each action's posterior and selects the highest sample. After observing the outcome (accuracy change minus a parameter cost penalty: $r = \Delta\text{acc} - 0.05 \cdot \Delta\text{params}/10^6$), it updates the chosen action's Beta parameters.

3.4 Stabilization Techniques (Ablated)

We also evaluate two optional stabilization techniques, which we ultimately find to be *counterproductive*:

Algorithm 1 BudgetNAS Online Loop

Require: Stream $\mathcal{D}$, budget B, model $\mathcal{M}$, controller C

```
for each domain d with known boundary do
    Replace classifier head for domain d
    Reset: B_rem ← B
    for each chunk D_t in domain d do
        Train M on D_t for E epochs
        Evaluate: acc_t
        if B_rem > 0 and acc_t < γ then
            mut* ← C.propose(acc_t, M)
            Apply mut* to M; B_rem ← B_rem − 1
        end if
    end for
end for
```

- **Warm-Start (WS)**: New blocks are initialized with Net2Net-style identity mappings [5] (second convolution weights set to zero so the residual path dominates initially).
- **Layer Freezing (FZ)**: After mutation, all blocks except the last two are frozen for one chunk, then unfrozen.

As shown in Section 4.3, both techniques reduce accuracy compared to random initialization, likely because identity-initialized blocks contribute too little during the critical first training chunk after mutation.

4 Experiments

4.1 Setup

Streaming Benchmark. We construct a task-incremental streaming benchmark: CIFAR-10 (10 classes) → CIFAR-100 (100 classes) → SVHN (10 digit classes). Each dataset is split into 4 sequential chunks, yielding 12 streaming steps. We subsample 8K training and 2K test examples per dataset for computational tractability. All experiments use three random seeds (42, 123, 7) and we report mean ± std.

Methods Compared (10 total). We compare ten methods spanning four categories:

Fixed-architecture baselines: **(1) Fixed**: 3-block ResNet retrained per domain. **(2) EWC** [9]: Fixed architecture with elastic weight consolidation (λ=400). **(3) Exp. Replay** [4]: Fixed architecture with 300-example replay buffer.

Unconditional growth: **(4) Growing**: Adds 2 blocks at every domain transition. **(5) DEN-style** [23]: Adds 1 block per domain, freezes early blocks. **(6) Smart Growing**: Adds blocks only when accuracy < 0.35, with warm-start.

Random baseline: **(7) RandomNAS** (B=1): Random mutation operator under the same budget.

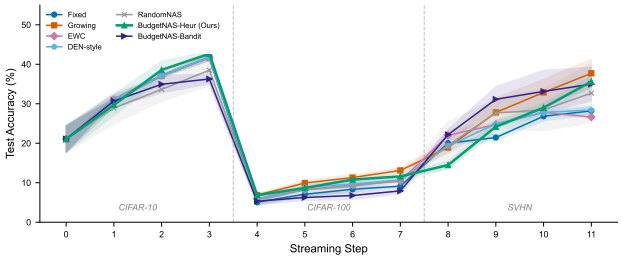

Figure 2: **Test accuracy over streaming steps** (mean ± std, 3 seeds, key methods shown). BudgetNAS-Heur (green, ▲) consistently tracks or exceeds baselines while using fewer parameters than Growing.

BudgetNAS variants: **(8) BudgetNAS-Heur** (B=1): Heuristic controller, no stabilization. **(9) BudgetNAS+WS+FZ** (B=1): Heuristic controller with warm-start and freezing. **(10) BudgetNAS-Bandit** (B=1): Thompson sampling controller.

Training. All models use SGD with momentum 0.9, weight decay 10^{-4}, learning rate 0.02, and cosine annealing. Each chunk is trained for 3 epochs with batch size 256.

4.2 Main Results

Table 1 reports final accuracy (mean ± std over 3 seeds) on each dataset, along with model size and the efficiency metric *Gain/M* (mean accuracy divided by maximum parameter count in millions).

Key Findings. (1) **BudgetNAS-Heur is the best architecture-changing method by efficiency.** It achieves the highest CIFAR-10 accuracy (42.7%, surpassing even fixed-architecture methods), second-highest CIFAR-100 (11.6%), and strong SVHN (35.6%) with only 1.47M parameters. Its Gain/M of 20.4 is the highest among all methods that modify architecture. (2) **Growing dominates on raw SVHN accuracy** (37.7%) but at the cost of 1.77M parameters and zero selectivity. (3) **RandomNAS has extreme variance**: 7.5% std on SVHN vs. 1.3% for BudgetNAS-Heur—a 5.8× variance reduction. (4) **Stabilization and bandit control both underperform** the simple heuristic (detailed in Section 4.3).

4.3 Ablation Studies

Budget Sweep ($B \in \{0, 1, 2, 3, 5\}$). We sweep the mutation budget using the bandit controller on seed 42 (Figure 4). B=0 (no mutations) yields a strong CIFAR-10 baseline (41.9%) but poor SVHN (24.2%). B=1 provides a large SVHN gain (+6.4%). Beyond B=2, SVHN accuracy plateaus while CIFAR-10 degrades as early mutations disrupt the initial architecture. At B=5, SVHN accuracy *drops*

Table 1: **Final accuracy (%, mean ± std, 3 seeds), model size, and efficiency.** Gain/M = mean final accuracy / max params (M). Best per column in **bold**, second-best underlined. BudgetNAS-Heur achieves the best accuracy–efficiency tradeoff among architecture-changing methods.

	Method	CIFAR-10	CIFAR-100	SVHN	Max Params	Blocks	Muts	Gain/M
Fixed	Fixed Backbone	41.7 ± 0.8	9.1 ± 0.1	28.2 ± 1.0	0.60M	3	0	44.1
	EWC	41.7 ± 0.8	10.4 ± 0.4	26.6 ± 0.3	0.60M	3	0	44.0
	Exp. Replay	41.7 ± 0.8	8.6 ± 0.3	27.6 ± 0.8	0.60M	3	0	43.5
Growth	Growing (Naive)	41.7 ± 0.8	$\mathbf{13.1} \pm 0.7$	$\underline{37.7 \pm 3.6}$	1.77M	7	4	17.5
	DEN-style	41.7 ± 0.8	10.7 ± 0.5	28.4 ± 1.0	1.18M	5	2	22.9
	Smart Growing	39.0 ± 4.1	10.9 ± 0.4	31.1 ± 1.1	2.36M	9	6	11.5
	RandomNAS (B=1)	38.5 ± 4.0	10.6 ± 0.5	32.7 ± 7.5	3.87M	6	3	7.0
Ours	**BudgetNAS-Heur** (B=1)	$\mathbf{42.7} \pm 0.7$	$\underline{11.6 \pm 0.5}$	35.6 ± 1.3	1.47M	6	3	**20.4**
	BudgetNAS+WS+FZ (B=1)	37.2 ± 2.0	8.9 ± 0.2	26.7 ± 1.6	1.47M	6	3	16.5
	BudgetNAS-Bandit (B=1)	36.3 ± 1.3	7.9 ± 0.8	$\mathbf{35.0} \pm 4.3$	2.98M	6	3	8.9

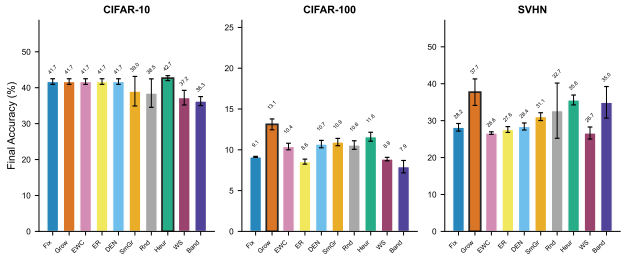

Figure 3: **Final accuracy per dataset** (mean ± std, 3 seeds, all 10 methods). BudgetNAS-Heur ("Heur") is competitive across all domains with low variance.

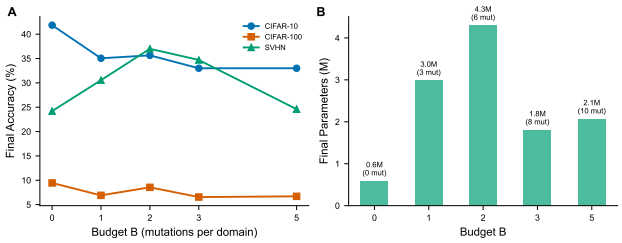

Figure 4: **Budget sweep. (A)** Final accuracy vs. budget B. B=1–2 provides the best tradeoff; $B \geq 3$ degrades. **(B)** Parameter count grows with B.

back to 24.6%—worse than B=0—demonstrating that excessive mutation is actively harmful. This confirms that B=1 is the practical sweet spot: enough capacity adaptation without optimization disruption.

Trigger Ablation. We compare four mutation trigger conditions (seed 42, B=1, with warm-start): *drift-only* (MMD-based drift detection), *acc-only* (accuracy < 0.35), *both*, and *neither* (Figure 5).

The result is striking: **drift detection produces zero mutations**. The MMD-based detector never fires across the entire stream, making "drift-only" identical to "neither" (both yield 0 mutations and Fixed-equivalent accuracy). All mutations in the "both" condition come from the accuracy trigger, making it identical to "acc-only." This finding simplifies the BudgetNAS controller: drift detection can be safely removed with no loss of functionality.

Stabilization Ablation. We compare four stabilization configurations (seed 42, B=1, heuristic controller): *none* (random init), *warm-start only*, *freeze only*, and *both* (Figure 6).

Counterintuitively, **no stabilization is best**: None (42.4%, 11.8%, 37.0%) > WS (39.0%, 11.1%, 33.1%) > FZ (37.4%, 10.1%, 28.7%) > WS+FZ (35.0%, 8.6%, 25.4%). Each stabilization technique monotonically *degrades* performance. We hypothesize that Net2Net identity initialization (zeroing the second convolution) makes new blocks too passive: they contribute minimally during the critical first training chunk, and the limited training budget (3 epochs per chunk) is insufficient for them to recover. Layer freezing compounds this by preventing early blocks from co-adapting with the new block. This is a **negative result** that contradicts the common assumption that function-preserving initialization helps in online settings with limited training.

4.4 Gradual Shift

We evaluate on a *gradual shift* stream where blended transition chunks (50/50 mix) are inserted between domains (Figure 7). BudgetNAS-Heur reaches 33.3% on the final domain vs. 29.3% for Fixed (+4.0% absolute). The bandit controller underperforms both (24.9%), again confirming

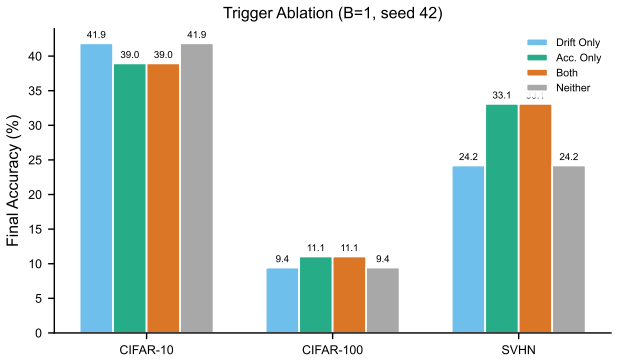

Figure 5: **Trigger ablation.** Drift-only = neither = 0 mutations (drift never fires). Acc-only = both = 3 mutations. All benefit comes from the accuracy trigger.

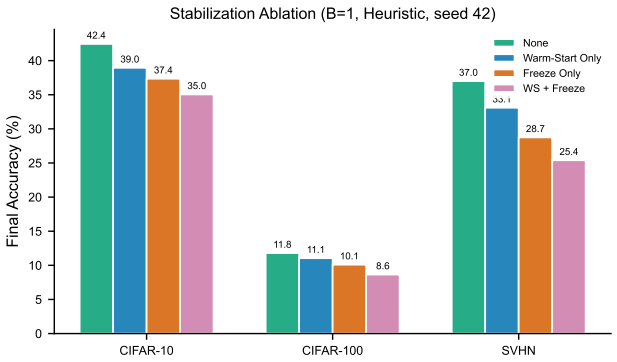

Figure 6: **Stabilization ablation.** Counterintuitively, no stabilization ("None") outperforms all stabilization variants. Net2Net identity initialization makes new blocks too passive during the critical first training chunk.

that the heuristic is more reliable at low budgets.

4.5 Long Stream (4 Domains)

We extend to a four-domain stream: CIFAR-10 → CIFAR-100 → SVHN → FashionMNIST (seed 42, Figure 8). Growing achieves the highest FashionMNIST accuracy (78.1%) by accumulating 7 blocks, while BudgetNAS-Bandit reaches 69.8%. RandomNAS achieves 79.9% on FashionMNIST but with 3.87M parameters. This experiment reveals that BudgetNAS's conservative mutation strategy (B=1) may under-adapt on longer streams where more capacity is genuinely needed, suggesting that adaptive budget schedules deserve investigation.

4.6 Efficiency Analysis

Figure 9 visualizes the accuracy–efficiency tradeoff. Fixed-architecture methods cluster at high Gain/M (low params, moderate accuracy). Growing and Smart Growing trade

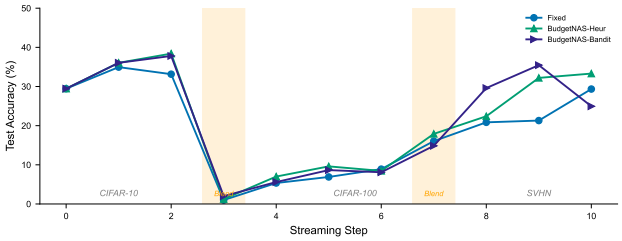

Figure 7: **Gradual shift stream.** Orange bands mark blended transition chunks. BudgetNAS-Heur outperforms Fixed by +4.0% on the final domain.

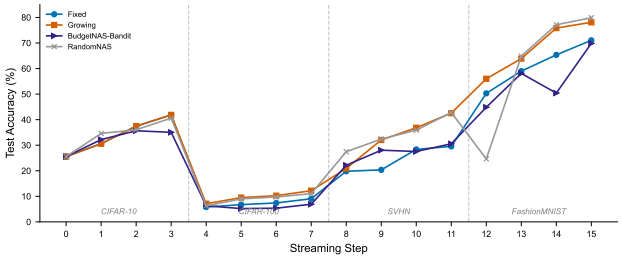

Figure 8: **Long stream (4 domains).** Growing and RandomNAS benefit from accumulated capacity on FashionMNIST; BudgetNAS-Bandit is competitive but constrained by B=1.

efficiency for accuracy. BudgetNAS-Heur occupies the *Pareto frontier*—higher accuracy than fixed methods at only 2.5× the parameter cost, and comparable accuracy to Growing at 17% fewer parameters.

5 Discussion

Why the Heuristic Beats the Bandit. With B=1, the bandit controller observes only 3 mutation outcomes across the entire stream—far too few for Thompson sampling to learn meaningful action preferences. The heuristic's hard-coded bias toward AddBlock happens to be near-optimal for this benchmark, while the bandit wastes its limited budget on exploratory AddDownsample operations that increase parameters without proportional accuracy gains. At higher budgets (B≥3), the bandit may have enough observations to learn, but higher budgets themselves degrade performance (see budget sweep). This creates a fundamental tension: the regime where budgeted mutation is most useful (B=1) is precisely where learned controllers have the least data.

Why Stabilization Hurts. The failure of warm-start contradicts Net2Net's success in offline transfer [5]. The key difference is training budget: Net2Net assumes extensive fine-tuning after morphism, while our streaming setting provides only 3 epochs per chunk. Identity-initialized blocks

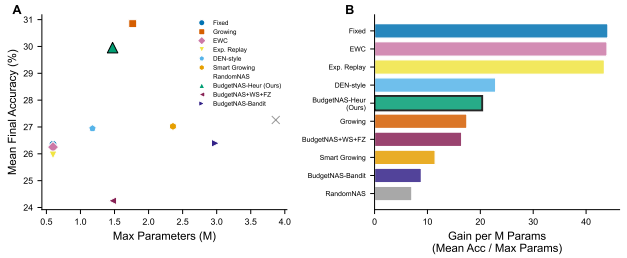

Figure 9: **Efficiency analysis.** **(A)** Accuracy vs. parameters scatter. **(B)** Gain/M ranking. BudgetNAS-Heur (▲) achieves the best tradeoff among architecture-changing methods.

act as near-no-ops for the first chunk, effectively wasting 25% of the domain's training on a block that barely contributes. Random initialization, while noisier, allows the new block to immediately participate in gradient flow.

Deployment Recommendation. Based on our experiments, we recommend the following configuration for practitioners deploying BudgetNAS:

1. **Controller**: Heuristic with performance trigger ($\gamma = 0.35$).
2. **Budget**: $B = 1$ per domain.
3. **Initialization**: Random (not warm-start).
4. **Freezing**: None.
5. **Drift detection**: Not needed.
6. **Preferred operator**: AddBlock (over AddDownsample).

Limitations. (1) The benchmark uses subsampled small-scale datasets; scaling to ImageNet remains open. (2) The heuristic's mutation priors are manually specified; sensitivity analysis across hyperparameter ranges would strengthen conclusions. (3) On the long stream, B=1 under-adapts for FashionMNIST, suggesting adaptive budgets may be needed for many-domain scenarios. (4) All methods share the same ResBlock architecture; heterogeneous blocks (e.g., attention, depthwise separable) could improve the mutation action space. (5) The task-incremental setting with known boundaries is simpler than class-incremental or boundary-free settings.

6 Conclusion

We introduced BudgetNAS, a framework for continual online NAS with explicit per-domain mutation budgets. Evaluating ten methods across three seeds with five ablation studies, we find that a simple heuristic controller with B=1 achieves the best accuracy–efficiency tradeoff: competitive accuracy with Growing at 17% fewer parameters, 5.8× lower variance than RandomNAS, and the highest Gain/M among architecture-changing methods. Our ablations reveal that several intuitively appealing design choices—drift detection, warm-start, layer freezing, learned controllers—are unnecessary or harmful at low mutation budgets. These negative results simplify the recommended BudgetNAS configuration to a single, easily-implemented rule: *if accuracy is low, add one block*. We believe this simplicity is a strength: BudgetNAS requires no drift detectors, no function-preserving initialization, and no bandit learning—just performance monitoring and restrained architectural growth.

References

[1] Han Cai, Chuang Gan, Tianzhe Wang, Zhekai Zhang, and Song Han. Once-for-all: Train one network and specialize it for efficient deployment. In *International Conference on Learning Representations (ICLR)*, 2020.

[2] Han Cai, Ligeng Zhu, and Song Han. ProxylessNAS: Direct neural architecture search on target task and hardware. In *International Conference on Learning Representations (ICLR)*, 2019.

[3] Olivier Chapelle and Lihong Li. An empirical evaluation of Thompson sampling. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2011.

[4] Arslan Chaudhry, Marcus Rohrbach, Mohamed Elhoseiny, Thalaiyasingam Ajanthan, Puneet K. Dokania, Philip H. S. Torr, and Marc'Aurelio Ranzato. On tiny episodic memories in continual learning. In *arXiv preprint arXiv:1902.10486*, 2019.

[5] Tianqi Chen, Ian Goodfellow, and Jonathon Shlens. Net2Net: Accelerating learning via knowledge transfer. In *International Conference on Learning Representations (ICLR)*, 2016.

[6] Thomas Elsken, Jan Hendrik Metzen, and Frank Hutter. Efficient multi-objective neural architecture search via Lamarckian evolution. In *International Conference on Learning Representations (ICLR)*, 2019.

[7] Thomas Elsken, Jan Hendrik Metzen, and Frank Hutter. Neural architecture search: A survey. *Journal of Machine Learning Research*, 20(55):1–21, 2019.

[8] Mustafa Burak Gurbuz and Constantine Dovrolis. NISPA: Neuro-inspired stability-plasticity adaptation for continual learning in sparse networks. In *International Conference on Machine Learning (ICML)*, 2022.

[9] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Charles Blundell, Daan Wierstra, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. In *Proceedings of the National Academy of Sciences (PNAS)*, volume 114, pages 3521–3526, 2017.

[10] Xilai Li, Yingbo Zhou, Tianfu Wu, Richard Socher, and Caiming Xiong. Learn to grow: A continual structure learning framework for overcoming catastrophic forgetting. In *International Conference on Machine Learning (ICML)*, 2019.

[11] Hanxiao Liu, Karen Simonyan, and Yiming Yang. DARTS: Differentiable architecture search. In *International Conference on Learning Representations (ICLR)*, 2019.

[12] David Lopez-Paz and Marc'Aurelio Ranzato. Gradient episodic memory for continual learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[13] Arun Mallya and Svetlana Lazebnik. PackNet: Adding multiple tasks to a single network by iterative pruning. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

[14] Hieu Pham, Melody Y. Guan, Barret Zoph, Quoc V. Le, and Jeff Dean. Efficient neural architecture search via parameter sharing. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018.

[15] Mingxing Tan, Bo Chen, Ruoming Pang, Vijay Vasudevan, Mark Sandler, Andrew Howard, and Quoc V. Le. MnasNet: Platform-aware neural architecture search for mobile. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

[16] Mingxing Tan and Quoc V. Le. EfficientNet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019.

[17] William R. Thompson. On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. *Biometrika*, 25(3/4):285–294, 1933.

[18] Tom Veniat, Ludovic Denoyer, and Marc'Aurelio Ranzato. Efficient continual learning with modular networks and task-driven priors. In *International Conference on Learning Representations (ICLR)*, 2021.

[19] Tao Wei, Changhu Wang, Yong Rui, and Chang Wen Chen. Network morphism. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, 2016.

[20] Mitchell Wortsman, Vivek Ramanujan, Rosanne Liu, Aniruddha Kembhavi, Mohammad Rastegari, Jason Yosinski, and Ali Farhadi. Supermasks in superposition. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[21] Bichen Wu, Xiaoliang Dai, Peizhao Zhang, Yanghan Wang, Fei Sun, Yiming Wu, Yuandong Tian, Peter Vajda, Yangqing Jia, and Kurt Keutzer. FBNet: Hardware-aware efficient convnet design via differentiable neural architecture search. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

[22] Tien-Ju Yang, Andrew Howard, Bo Chen, Xiao Zhang, Alec Go, Mark Sandler, Vivienne Sze, and Hartwig Adam. NetAdapt: Platform-aware neural network adaptation for mobile applications. In *European Conference on Computer Vision (ECCV)*, 2018.

[23] Jaehong Yoon, Eunho Yang, Jeongtae Lee, and Sung Ju Hwang. Lifelong learning with dynamically expandable networks. In *International Conference on Learning Representations (ICLR)*, 2018.

[24] Jiahui Yu, Linjie Yang, Ning Xu, Jianchao Yang, and Thomas S. Huang. Slimmable neural networks. In *International Conference on Learning Representations (ICLR)*, 2019.

[25] Barret Zoph and Quoc V. Le. Neural architecture search with reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2017.